



Production Low-Level Design and Implementation Contract

Final Updated Version - All Latest Changes Included

Text-first clinic receptionist engine, admin/doctor application, voice-ready architecture, appointment capacity engine, and production DB schema catalog

Area	Final decision	Reason
Backend stack	NestJS + TypeScript using Fastify adapter	Final backend decision for scratch build. Gives modular architecture, DI, guards, queues, WebSocket readiness, shared TypeScript with Next.js, and production maintainability.
Frontend stack	Next.js + TypeScript + Tailwind CSS	Best suited for role-based admin/doctor portal, protected routes, layouts, and future SaaS growth.
Database	PostgreSQL as source of truth	All clinic, appointment, slot, patient, call, knowledge, audit, and notification data lives in Postgres. Redis is not the source of truth.
ORM / migrations	Drizzle ORM + SQL migrations	SQL-first, supports composite FKs, exclusion constraints, raw SQL migrations, pgvector later, and strong typed queries.
Auth	OTP login through Supabase Auth or equivalent JWT provider	Login UI is lower priority, but backend RBAC and membership design must exist from the start.
Queue / Redis	BullMQ + Redis/Valkey in production; in-memory/inline fallback in local	Used for background jobs, notifications, cleanup, slot generation, and locks. Can be adapted later through interfaces.
Voice	Voice comes after text QA. Provider adapter pattern.	Exotel likely first India pilot, Twilio possible. Do not hardcode provider.
LLM	Sarvam 30B primary, Sarvam 105B fallback through adapter	LLM classifies/extracts only. Backend validates and acts.
Booking	Default pending confirmation; auto-confirm optional by admin config	Keeps clinic in control during pilot.
Slots	Capacity-window model	appointment_slots = time window + capacity; slot_holds = temporary reservation; appointment_requests = actual booking/request.
Retention	Audio 10 days; transcript 30 days	Call audio stored in object storage only. Transcript text has longer retention.

Table of Contents

1. Purpose and Latest Change Summary
2. Core Architecture and Principles
3. User, Clinic, Doctor, Patient and Onboarding Model
4. Auth and Login Flow
5. Agent Enabled, Call Routing, and Missed Call Behavior
6. Redis, Queue, and Background Jobs
7. API Method Rules: POST, PUT, PATCH
8. Appointment Slot Capacity Engine

9. Slot Generation Policy and Schedule Changes
10. Manual Appointment, Edit Thresholds, Cancel/Rebook
11. Agent Flow and Booking Flow
12. Application Design and Permissions
13. Knowledge Base and Language Extensibility
14. Subscriptions and Plan Controls
15. Notification, Recording, and Transcript Rules
16. Complete Database Schema Catalog
17. API Contract Summary
18. State Transitions, Test Matrix, and Readiness Checklist

1. Purpose and Latest Change Summary

This document is the final updated production low-level design for Vaidya. It includes the earlier architecture decisions and all latest corrections discussed in the recent design review. It is intended to be detailed enough for other AI agents or developers to build from without inventing missing behavior.

The most important new update is the appointment capacity model: appointment slots are not single bookings. A slot is a doctor-service time window with capacity. Temporary selection is stored in slot_holds. Actual patient appointment/request is stored in appointment_requests.

Topic	Final update / clarification
Appointment slots	appointment_slots represent bookable time windows with capacity_total, not one appointment. Slot status should be open/blocked/cancelled/superseded, not booked.
Slot holds	slot_holds table is required. Holds are created only after patient selects a proposed slot. Hold expires after a short time, default 5 minutes.
Concurrent bookings	Prevented through Postgres transaction and SELECT FOR UPDATE on appointment_slots. Availability is capacity_total minus active appointments minus active non-expired holds.
Doctor-service booking rules	New table required to store slot duration, capacity per slot, booking horizon, min notice, edit cutoff, and max shift per doctor-service mapping.
45-day slot generation	Daily SlotGenerationJob maintains rolling 45-day future slots for every active doctor-service mapping.
Schedule/rule changes	Regenerate only safe future open slots. Do not auto-cancel. Block or show conflict list if future pending/confirmed appointments or active holds conflict.
Manual edit cutoff	Add manual_edit_cutoff_before_start_minutes and manual_edit_max_shift_minutes. More than allowed shift uses formal cancel/rebook or reschedule flow.
API methods	POST creates resources/actions, PUT replaces full resources/collections, PATCH partially updates fields.
Clinic settings APIs	clinic_settings is singleton created with clinic. Use GET/PATCH, optional PUT; no normal POST.
Schedules and clinic hours APIs	Use GET + PUT full replacement + POST add window + PATCH edit window + disable action.
Login	OTP based login is enough. Login UI is lower priority, but backend user/clinic_users/RBAC design must exist early.
Users and clinic_users	users are login identities. clinic_users is clinic-specific membership/role mapping. doctors are clinical profiles and can exist without login.
Admin user management	Clinic admin can enable/disable clinic users by setting clinic_users.active. Platform admin controls global users.
Agent enabled	Store in clinic_settings.agent_enabled. If false, voice call forwards to fallback_phone. Agent does not automatically call back a disconnected missed call in v1.
Redis and queue	Redis is not source of truth. Used for BullMQ jobs, notifications, cleanup, slot generation, hold expiry, DOCX parsing, embeddings later, reports later, locks/rate limits.
Subscriptions	Add subscription_plans, clinic_subscriptions, clinic_usage_monthly, subscription_events. Least priority, but schema and plan enforcement design should exist.
Languages	Add supported_languages, clinic_languages, message_templates, language pack design, knowledge translations. New language should not require rewriting state machines.

2. Core Architecture and Principles

Vaidya is a modular monolith at first. Application servers are stateless. PostgreSQL stores durable truth. Redis/Valkey is used for background jobs and temporary infrastructure, not business truth. Object storage stores call recordings.

Patient / Admin / Doctor

- > Next.js Web App / Text Test Console / Voice Adapter
- > NestJS API Layer - Fastify adapter, Zod validation, RBAC guards
- > Domain Services - conversation, appointment, slot, knowledge, calls, notifications, audit
- > PostgreSQL + Redis/Valkey + Object Storage

- Text first: All booking, cancel, reschedule, handoff, knowledge, safety, and structured lookup flows are tested as text before voice.
- Voice later: voice is only a channel wrapper around the same text-core engine. STT/TTS do not own business logic.
- LLM never writes to DB and never deletes records.
- Backend state machines own conversation progression.
- ActionValidator gates every write action.
- Structured DB wins over knowledge base for fees, timings, schedules, holidays, location, services, and slots.
- Approved knowledge only: if missing, Vaidya says clinic staff will confirm.
- No hard delete in normal UI. Use disabled, cancelled, archived, expired, superseded statuses.
- All critical actions write audit logs with actor, clinic, source, old/new values, and request_id.

3. User, Clinic, Doctor, Patient and Onboarding Model

The application separates login identity from clinic membership and clinical doctor profile. This prevents permission confusion and supports doctors who do not log in.

Concept	Purpose	Important rule
users	Global login identities for clinic admins, doctors, platform admins, and future receptionist users.	Patients are not users. A doctor may not have a user row until invited to log in.
clinic_users	Clinic-specific membership and role mapping.	A user can belong to one or more clinics. role and doctor_id live here.
doctors	Clinical doctor profile used for schedules, services, slots, fees, appointments, and routing.	Doctor can exist without login. user_id may be null.
patients	Clinic patients seen through appointments and visits.	Patients are identified cautiously using clinic_id + normalized phone + normalized name.

3.1 Why platform admin creates clinic first

For the first production version, clinic creation should be controlled by the platform team. Clinic creation is tenant creation. It creates clinic record, default clinic_settings, initial admin membership, plan/subscription later, and safe defaults. This prevents duplicate clinics, wrong fallback numbers, wrong ownership, and agent activation before setup is complete.

1. Platform admin creates clinics row with name, phone, address, timezone, default language, active status.
2. System creates clinic_settings row with safe defaults: agent_enabled=false, answering_mode=off, booking_mode=pending_confirmation, fallback_phone set to old clinic/reception number, retention values, max_concurrent_calls based on plan.
3. Platform admin creates or invites first clinic admin user.
4. System creates clinic_users membership with role=clinic_admin and active=true.
5. Clinic admin completes doctors, services, schedules, booking rules, fees, knowledge, holidays, and fallback phone setup.
6. Only after setup is reviewed does clinic admin enable the agent.

3.2 Doctor created without login and later login mapping

When clinic admin adds a doctor, backend creates a doctors row immediately. The doctor_id is generated at that time even if no login user exists.

```
Before doctor login:
doctors.id = doctor_001
doctors.clinic_id = clinic_001
doctors.name = Dr. Priya
doctors.user_id = null
doctors.active = true
```

Admin can still configure schedules, services, fees, slots, and appointments for doctor_001.

If the doctor later needs app login, a users row is created and linked to the existing doctor profile using clinic_users.doctor_id. The user ID is the login identity; the doctor ID remains the clinical profile.

```

After doctor login invite:
users.id = user_002
users.name = Dr. Priya

clinic_users.clinic_id = clinic_001
clinic_users.user_id = user_002
clinic_users.role = doctor
clinic_users.doctor_id = doctor_001
clinic_users.active = true

```

When Dr. Priya logs in, RBACGuard loads clinic_users and allows only doctor_001 appointments, timings, and services.

3.3 Enable/disable users

- Clinic admin can disable a user for that clinic by setting clinic_users.active=false.
- Disabling clinic_users prevents login access to that clinic but does not disable the doctor profile.
- Disabling doctors.active=false stops routing/bookings/slot generation for that doctor.
- Platform admin can globally disable users.active=false across all clinics.
- Do not hard-delete users, clinic_users, or doctors in normal UI.

4. Auth and Login Flow

Login UI is lower priority than application workflows, but backend RBAC must be designed early. Use passwordless OTP login through Supabase Auth or equivalent provider. OTP verifies identity; it does not mean open public self-signup.

Endpoint	Purpose	Priority
POST /v1/auth/request-otp	Send OTP to email or phone for an invited/known user.	Before QA login
POST /v1/auth/verify-otp	Verify OTP and return access token, refresh token, user, memberships.	Before QA login
GET /v1/me	Return current user, clinic memberships, roles, doctor_id, permissions.	Early backend
POST /v1/auth/logout	End session.	Before production

During early local development, AUTH_MODE=dev may be used with seeded admin and doctor users. This must never be enabled in production.

```

verify-otp response shape:
{
  "access_token": "...",
  "refresh_token": "...",
  "user": { "id": "...", "name": "Dr. Priya", "email": "...", "phone": "... },
  "clinics": [
    { "clinic_id": "...", "clinic_name": "Sri Murugan Clinic", "role": "doctor", "doctor_id":
    "..."}
  ]
}

```

5. Agent Enabled, Call Routing, and Missed Call Behavior

agent_enabled is stored in clinic_settings. The voice adapter checks this setting before answering calls. If agent_enabled=false, the call should be forwarded to fallback_phone through the telephony provider.

Mode	Behavior
always_on	Vaidya answers incoming configured calls directly.
after_hours_only	Vaidya answers only outside clinic hours or holidays.
overflow_after_n_rings	Clinic phone rings first. If not answered after configured rings, provider forwards to Vaidya.
holiday_only	Vaidya answers only on configured clinic holidays.
off	Vaidya does not answer. Calls forward to fallback_phone.

Important missed call clarification: Vaidya can talk automatically only if the telephony provider forwards the live call to Vaidya. If the patient already disconnected before forwarding, Vaidya cannot speak in that call. Later missed-call recovery may create callback/SMS/outbound workflow, but outbound automatic calling is not part of v1.

6. Redis, Queue, and Background Jobs

Redis/Valkey is not the source of truth. PostgreSQL remains the durable store. Redis is used to run reliable background jobs, repeat jobs, distributed locks, rate limits, provider retries, and future voice event buffering.

Use case	Why queue/Redis is used	Source of truth
WhatsApp/SMS notification	Provider can be slow or fail. Queue retries without blocking API.	notification_events in Postgres
Emergency alert	High-priority notification with retry, while patient gets immediate fixed 108 response.	emergency_incidents + notification_events
Callback request alert	Notify admin/receptionist asynchronously.	callback_requests + notification_events
Delete expired recordings	Object storage deletion may fail and should retry.	calls recording metadata
Delete expired transcripts	Batch cleanup should run outside API requests.	call_transcripts
Expire slot holds	Repeat job every 1 minute to mark stale active holds expired.	slot_holds
Generate future slots	Daily rolling 45-day generation can be slow for many clinics/doctors.	doctor_service_booking_rules + appointment_slots
Parse uploaded DOCX	File parsing may be slow. Create pending_review rows asynchronously.	knowledge_files + clinic_knowledge_base
Generate embeddings later	Embedding provider is slow/retryable and not required for immediate approval UI.	clinic_knowledge_base vector fields
Daily clinic report later	Scheduled report generation and delivery.	calls/appointments/notifications

Redis can be adapted later if code uses QueueService, LockService, SchedulerService, and RateLimitService interfaces. Local development may use in-memory locks and inline/no-op queues. Production should use BullMQ + Redis/Valkey.

Slot booking correctness must not depend on Redis. Slot capacity and double-booking prevention must be guaranteed by Postgres transactions and row locks.

7. API Method Rules: POST, PUT, PATCH

Method	Use when	Example
POST	Create a new resource or trigger an action. Not idempotent by default unless idempotency_key is used.	POST /v1/doctors, POST /v1/holidays, POST /v1/appointments/manual
PUT	Replace a full resource or full collection. Missing items may be removed/disabled based on contract.	PUT /v1/doctors/{id}/schedules, PUT /v1/clinic-hours
PATCH	Partially update existing resource fields without replacing the whole object.	PATCH /v1/clinic/settings with { "agent_enabled": false }

Resource	Recommended APIs
clinic_settings	GET /v1/clinic/settings; PATCH /v1/clinic/settings; optional PUT /v1/clinic/settings. No normal POST because settings row is created with clinic.
doctors	GET /v1/doctors; POST /v1/doctors; GET /v1/doctors/{doctorId}; PATCH /v1/doctors/{doctorId}; POST /v1/doctors/{doctorId}/disable.
doctor schedules	GET /v1/doctors/{doctorId}/schedules; PUT full weekly replacement; POST add one window; PATCH edit one window; POST disable one window.
clinic hours	GET /v1/clinic-hours; PUT full weekly replacement; POST add one window; PATCH edit one window; POST disable one window.
holidays	GET /v1/holidays; POST /v1/holidays; PATCH /v1/holidays/{id}; POST /v1/holidays/{id}/disable.
clinic services	GET /v1/clinic-services; POST create; PATCH edit; POST disable.
doctor-service mappings	GET /v1/doctor-services; POST create mapping; PATCH edit mapping; POST disable mapping.

doctor-service booking rule	GET /v1/doctors/{doctorId}/services/{serviceId}/booking-rule; PUT create/replace full rule; PATCH partial update.
-----------------------------	---

8. Appointment Slot Capacity Engine

Final master rule: a slot is not a single appointment. A slot is a doctor-service time window with capacity. A patient booking is an appointment_request. A temporary reservation is a slot_hold.

Table	Meaning	Example
appointment_slots	Supply: bookable doctor-service time window and capacity.	Dr. Kumar, Orthopaedic, 6:30-7:00, capacity_total=3.
slot_holds	Temporary reservation while patient is deciding/confirming.	Patient selected 6:30; hold active until now+5 minutes.
appointment_requests	Actual patient appointment/request.	Kumar, knee pain, Dr. Kumar, 6:30, pending_confirmation.

8.1 Availability formula

```
active_appointments = count appointment_requests where status in (pending_confirmation, confirmed)
active_holds = count slot_holds where status=active and hold_expires_at > now()
available_count = appointment_slots.capacity_total - active_appointments - active_holds

If available_count > 0, slot can be proposed/held. If available_count <= 0, slot is full.
```

8.2 Slot proposal vs hold

- Vaidya can propose multiple slots, but proposed slots are snapshots, not guarantees.
- Do not hold all proposed slots. This would unnecessarily block other patients.
- Create slot_hold only after patient chooses a specific slot.
- If proposed slot becomes full before patient chooses it, hold attempt fails and Vaidya offers alternatives.

8.3 Concurrent hold transaction

```
BEGIN;

SELECT * FROM appointment_slots
WHERE id = $slot_id AND clinic_id = $clinic_id AND status = 'open'
FOR UPDATE;

-- Count active appointments and active non-expired holds inside the same transaction.
-- If occupied_count < capacity_total, insert slot_holds row.
-- Else rollback and return SLOT_FULL.

COMMIT;
```

This prevents double booking even if two patients choose the same slot at the same time. The second transaction waits for the locked row and then sees updated occupancy.

8.4 Hold expiry

- Default hold time: 5 minutes.
- Availability queries ignore active holds whose hold_expires_at <= now, even if cleanup job has not run yet.
- SlotHoldExpiryJob runs every 1 minute and marks expired holds as status=expired.
- If patient confirms after hold expiry, backend re-checks capacity. If still available, it can create a new hold or create appointment transactionally. If not available, Vaidya asks for another slot.

9. Slot Generation Policy and Schedule Changes

Vaidya should pre-generate slots for the next 45 days for every active doctor-service mapping. Pre-generated slots make availability queries, dashboard display, conflict detection, and concurrency control much easier.

9.1 Doctor-service booking rules

doctor_service_booking_rules is required because slot duration and capacity are often different for each doctor-service combination, not just clinic-wide or doctor-wide.

Example	Rule
Dr. Murugan - General Consultation	slot_duration=15 minutes, capacity_per_slot=3
Dr. Murugan - Diabetes Follow-up	slot_duration=10 minutes, capacity_per_slot=4
Dr. Kumar - Orthopaedic Consultation	slot_duration=30 minutes, capacity_per_slot=1
Dr. Priya - Paediatric Consultation	slot_duration=15 minutes, capacity_per_slot=2

9.2 Daily rolling 45-day job

- Job name: SlotGenerationJob.
- Run daily at 1:00 AM in clinic timezone. For initial India-only deployment, Asia/Kolkata is acceptable; keep clinic timezone in DB for future.
- For each active clinic and active doctor_service mapping, load booking rule, doctor schedule, clinic holidays, doctor blocked slots, and active status.
- Generate missing slots from today through today + booking_horizon_days, default 45.
- Do not regenerate all rows daily. Insert only missing slots.
- Slots store generated_from_rule_id, generation_batch_id, and config_version for traceability.

9.3 Triggered regeneration on config changes

- Run impact preview on doctor schedule change, clinic holiday change, clinic hour change, doctor blocked slot change, doctor_service_booking_rules change, doctor_service activation/deactivation, clinic_service activation/deactivation, and doctor activation/deactivation.
- If no occupied future slots are affected, regenerate safe future slots.
- If conflicts exist, block change by default and return conflict list. Do not auto-cancel appointments.
- Admin may choose later effective_from date to avoid conflicts.

9.4 Capacity and duration changes

Change	If no conflicts	If conflicts
Capacity increases	Update capacity_total for future open slots.	Normally no conflict because capacity increased.
Capacity decreases	If occupancy <= new capacity, update capacity_total for future open slots.	If occupancy > new capacity, block change and show affected appointments/holds.
Duration increases/decreases	Mark old future open slots as superseded and generate new slots with new duration.	If affected slots have pending/confirmed appointments or active holds, block change and show conflicts.

Do not hard-delete old generated slots during regeneration. Mark unused future old rows as superseded. A later cleanup job can purge unused superseded rows if needed.

10. Manual Appointment, Edit Thresholds, Cancel/Rebook

Two separate thresholds are required: cutoff before appointment start and maximum shift allowed for quick edit.

Setting	Meaning	Default
manual_edit_cutoff_before_start_minutes	How close to appointment start time quick edit is blocked.	60
manual_edit_max_shift_minutes	How far the appointment time can be moved as a quick edit.	60

10.1 Doctor quick edit rules

- Doctor can edit only own appointments.
- Allowed only for pending_confirmation or confirmed appointments.
- Must remain same doctor and same service.
- New slot must have available capacity.
- Appointment must not be inside manual_edit_cutoff_before_start_minutes.
- Time shift must be $\leq$ manual_edit_max_shift_minutes.
- Doctor cannot override outside hours in v1.
- Doctor cannot edit cancelled, visited, or no_show appointments.

10.2 Admin quick edit rules

- Admin can edit clinic appointments subject to status and capacity rules.
- Admin can override outside clinic hours only with override_reason.
- Admin cannot override double booking, inactive doctor, inactive service, or invalid doctor-service relation.
- If time shift exceeds manual_edit_max_shift_minutes, use formal cancel/rebook or reschedule flow, not quick edit.

10.3 More-than-one-hour change

If appointment time is changed more than manual_edit_max_shift_minutes, product behavior should be cancel and rebook / formal reschedule. Production-preferred backend model is new appointment row + old appointment marked as rescheduled/cancelled_due_to_rebook, linked by replaces_appointment_id and replaced_by_appointment_id.

7. Hold new slot transactionally.
8. Create new appointment_request.
9. Mark old appointment as rescheduled or cancelled_due_to_rebook.
10. Link old and new appointment records.
11. Release old slot capacity by status change.
12. Create appointment_event.
13. Send patient notification after clinic confirms or after final action.

11. Agent Flow and Booking Flow

Vaidya greeting should ask for the problem first because many patients do not know doctor names.

Tanglish greeting:

Vanakkam. {clinic_name} Vaidya pesuren. Enna problem-ku appointment venum?

English greeting:

Hello. This is Vaidya from {clinic_name}. What problem do you need the appointment for?

- The agent must support compound messages and extract all provided details: problem, doctor, date, time, patient name, follow-up wording.
- If patient already visited and same problem is detected, prefer previous doctor only after patient identity, service, doctor active status, and doctor-service mapping are valid.
- If same phone has multiple patients and patient name is unknown, ask which patient the appointment is for before using history.
- If no history or different problem, use service router with active clinic_services and doctor_services.
- Side questions during active booking must be answered and the booking prompt resumed.
- Negative phrases such as vendam/no/stop/cancel should stop active flow and release hold, except cancel-flow confirmation where yes means cancel and no means do not cancel.
- Emergency and medical advice override all flows.

12. Application Design and Permissions

Tab	Admin	Doctor
Home	Full clinic summary, pending actions,	Own appointments and own pending actions

	callbacks, emergency alerts, agent status.	only.
Call inbox	Visible. Last calls, summaries, recordings, transcript, outcomes.	Not visible.
Appointments	All pending/confirmed/visited/action-request records.	Own appointments and own visited records only.
Clinic setup	Full setup including agent mode, schedules, hours, holidays, services, booking rules.	Own timings and own services only if allowed.
Knowledge base	Upload/use template, manual Q&A, approve/edit/disable.	No access initially.

Appointment cards must show patient name, phone, doctor, service, date/time, problem/reason_for_visit, new/follow-up badge, routing source, history indicator, and status.

Mark visited must require reason_for_visit. It creates patient_visits row and supports future follow-up routing.

13. Knowledge Base and Language Extensibility

Knowledge base is approved Q&A only. Operational facts come from structured DB. New language addition must be easy and must not require rewriting booking/cancel/reschedule state machines.

Data type	Source of truth
Doctor fee	doctor_services fee fields or dedicated fee table
Clinic timing	clinic_hours
Doctor availability	doctor_schedules + appointment_slots
Location	clinics table
Service routing	clinic_services + doctor_services
Holidays	clinic_holidays
Parking, scan prep, first visit documents, insurance explanation, report process	approved clinic_knowledge_base / translations

13.1 Language extensibility

- supported_languages stores platform languages.
- clinic_languages stores languages enabled for each clinic and default language.
- message_templates stores template_key + language_code + template_text + required variables.
- LanguagePack contains parser words for yes/no/cancel/date/time, classifier examples, STT hints, TTS voice config.
- Knowledge translations should be approved before runtime use. Do not freely LLM-translate medical/clinic instructions without approval.
- Adding a new language should require templates, parser words, classifier examples, STT/TTS settings, knowledge translations, and tests - not state-machine rewrites.

14. Subscriptions and Plan Controls

Subscription/billing is least priority for product UI, but schema and plan entitlement design should exist to avoid redesign. Subscription controls what the clinic is commercially allowed to use. clinic_settings controls actual behavior.

Concept	Purpose
subscription_plans	Defines Starter, Pro, Clinic+, Custom limits and entitlements.
clinic_subscriptions	Clinic current plan, status, billing period, trial, plan snapshot.
clinic_usage_monthly	Monthly voice minutes, calls, messages, appointments, recordings usage.
subscription_events	Plan changes, trial started, upgrades/downgrades, manual credits, payment failures.

Backend must enforce plan limits: max concurrent calls, voice enabled, recording retention, auto-confirm allowance, included minutes, and overage behavior. For first pilot, platform admin can assign plans manually; payment automation can come later.

15. Notification, Recording, and Transcript Rules

- Patient WhatsApp/SMS confirmation is sent only after appointment status becomes confirmed.
- Pending confirmation does not notify patient as confirmed.
- Optional staff action notification can be sent when pending_confirmation appointment is created, based on clinic setting.
- Appointment time edited by admin/doctor sends patient notification after final action.
- Appointment cancelled by admin/doctor sends patient notification.
- Patient cancel/reschedule by call creates admin action request and notifies admin; admin final action triggers patient notification.
- Emergency triggers fixed 108 patient response and clinic emergency alert notification.
- Recordings are stored in object storage for 10 days. Transcript text is retained for 30 days. Recording access uses signed URLs and audit logs.

Notification event	Recipient	When
APPOINTMENT_CONFIRMED_PATIENT	Patient	Appointment enters confirmed.
APPOINTMENT_TIME_CHANGED_PATIENT	Patient	Admin/doctor edits appointment time or formal reschedule completes.
APPOINTMENT_CANCELLED_PATIENT	Patient	Admin/doctor cancels appointment.
PENDING_APPOINTMENT_STAFF	Admin/reception staff	Optional when Vaidya creates pending_confirmation request.
PATIENT_CANCEL_REQUEST_ADMIN	Admin	Patient requests cancel by call/text.
PATIENT_RESCHEDULE_REQUEST_ADMIN	Admin	Patient requests reschedule by call/text.
CALLBACK_REQUEST_ADMIN	Admin	Callback request created.
EMERGENCY_ALERT_ADMIN	Admin/clinic alert contacts	Emergency incident created.
KNOWLEDGE_GAP_ADMIN	Admin	Patient asked question with no approved answer, optional later.

16. Complete Database Schema Catalog

The following schema catalog is the production data model. Actual migrations should be implemented as Drizzle migrations plus raw SQL for advanced constraints. All clinic-owned operational tables must include clinic_id and must never be queried without clinic_id filtering.

Type names are suggested PostgreSQL types. Enums can be PostgreSQL enums or text with check constraints. UUID default should use gen_random_uuid() from pgcrypto.

clinics

Tenant/clinic master record. Stores identity and basic clinic facts.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
name	text	no	Clinic display name.
primary_phone	text	yes	Main clinic number.
address_line1	text	yes	Structured address.
address_line2	text	yes	Optional address.
city	text	yes	City.
state	text	yes	State.
postal_code	text	yes	PIN code.
timezone	text	no	Default Asia/Kolkata.
default_language_code	text	no	Default ta_tanglish.
active	boolean	no	Tenant active/inactive.
created_at	timestampz	no	Created timestamp.
updated_at	timestampz	no	Updated timestamp.

Key constraints:

- id primary key
- unique clinic slug later if needed

Important indexes:

- index active clinics by active

Implementation notes:

- clinic_settings row must be created automatically with clinic.

clinic_settings

Singleton per clinic for agent behavior, booking mode, retention, and notification defaults.

Column	Type	Nullable	Purpose / notes
clinic_id	uuid	no	PK and FK to clinics.
agent_enabled	boolean	no	Admin on/off control.
answering_mode	text	no	always_on, after_hours_only, overflow_after_n_rings, holiday_only, off.
fallback_phone	text	yes	Old mobile/reception number used when agent off or transfer required.
overflow_after_rings	integer	yes	Used for overflow mode.
booking_mode	text	no	pending_confirmation or auto_confirm.
max_concurrent_calls	integer	no	Actual configured limit, must not exceed subscription plan.
recording_retention_days	integer	no	Default 10.
transcript_retention_days	integer	no	Default 30.
notify_staff_on_pending_appointment	boolean	no	Optional staff notification on pending appointment.
pending_appointment_notification_channel	text	yes	dashboard, whatsapp, sms, email.
allow_doctor_service_edit	boolean	no	Default false.
allow_patient_auto_cancel	boolean	no	Default false; patient calls create action request by default.
updated_by_user_id	uuid	yes	Last updater.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- clinic_id primary key references clinics(id)

Implementation notes:

- No normal POST API; row is created with clinic.

users

Global login identities for admins, doctors, platform admins, and future staff. Patients are not users.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
auth_provider_id	text	yes	Supabase Auth user id or provider id.
name	text	yes	Display name.
email	text	yes	Login email.
phone	text	yes	Login phone.
active	boolean	no	Global login active.
last_login_at	timestampz	yes	Last login.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique email where email is not null
- unique phone where phone is not null
- unique auth_provider_id where not null

Implementation notes:

- Clinic admin disables clinic access through clinic_users.active; platform admin controls users.active.

clinic_users

Clinic-specific membership and RBAC role mapping.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
user_id	uuid	no	FK users.
role	text	no	platform_admin, clinic_admin, doctor.
doctor_id	uuid	yes	Required for doctor role, FK doctors.
active	boolean	no	Membership active for this clinic.
invited_by_user_id	uuid	yes	Inviter.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique (clinic_id, user_id)
- if role=doctor then doctor_id not null
- doctor_id must belong to same clinic

Important indexes:

- index (clinic_id, role, active)

Implementation notes:

- Same user may belong to multiple clinics.

otp_challenges

Optional local OTP tracking if not fully delegated to auth provider.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
identifier	text	no	Email or phone.
otp_hash	text	no	Hash only; never store raw OTP.
purpose	text	no	login, invite, verify_phone.
expires_at	timestampz	no	Expiry.
consumed_at	timestampz	yes	When used.
attempt_count	integer	no	Rate limiting.
created_at	timestampz	no	Created.

Important indexes:

- index (identifier, purpose, expires_at)

Implementation notes:

- Can be skipped if Supabase Auth OTP is fully used.

subscription_plans

Commercial plan definitions and entitlements.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
plan_key	text	no	starter, pro, clinic_plus, custom.
name	text	no	Display name.
billing_cycle	text	no	monthly, yearly, custom.
currency	text	no	INR by default.
base_price	numeric(12,2)	no	Plan price.
included_voice_minutes	integer	yes	Included minutes.
max_concurrent_calls	integer	no	Plan limit.
max_doctors	integer	yes	Optional limit.
max_admin_users	integer	yes	Optional limit.
max_knowledge_items	integer	yes	Optional limit.
recording_retention_days	integer	no	Allowed audio retention.
transcript_retention_days	integer	no	Allowed transcript retention.
auto_confirm_allowed	boolean	no	Entitlement.
voice_enabled	boolean	no	Entitlement.
active	boolean	no	Plan available.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique plan_key

Implementation notes:

- Least priority, but schema should exist.

clinic_subscriptions

Current subscription for each clinic.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
subscription_plan_id	uuid	no	FK subscription_plans.
status	text	no	trialing, active, past_due, cancelled, expired, manual_free.
billing_provider	text	yes	manual, razorpay, stripe, etc later.
external_customer_id	text	yes	Provider id.
external_subscription_id	text	yes	Provider id.
current_period_start	date	yes	Billing period.
current_period_end	date	yes	Billing period.
trial_start	date	yes	Trial start.
trial_end	date	yes	Trial end.
cancel_at	date	yes	Scheduled cancel.
cancelled_at	timestampz	yes	Cancelled at.
plan_snapshot_json	jsonb	no	Snapshot of entitlements when assigned.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Important indexes:

- index (clinic_id, status)

Implementation notes:

- Manual assignment is fine for pilot.

clinic_usage_monthly

Monthly usage aggregation for billing and plan controls.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
billing_month	date	no	First day of month.
voice_call_count	integer	no	Calls.
voice_minutes_used	numeric(12,2)	no	Minutes.
stt_minutes_used	numeric(12,2)	no	STT minutes.
tts_characters_used	integer	no	TTS chars.
sms_sent_count	integer	no	SMS count.
whatsapp_sent_count	integer	no	WhatsApp count.
appointment_requests_created	integer	no	Requests.
confirmed_appointments	integer	no	Confirmed.
callback_requests_created	integer	no	Callbacks.
recording_storage_gb	numeric(12,4)	no	Storage estimate.
overage_voice_minutes	numeric(12,2)	no	Overage.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique (clinic_id, billing_month)

subscription_events

Audit/history for subscription changes.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
clinic_subscription_id	uuid	yes	FK clinic_subscriptions.
event_type	text	no	trial_started, plan_upgraded, payment_failed, etc.
old_plan_id	uuid	yes	Old plan.
new_plan_id	uuid	yes	New plan.
event_data_json	jsonb	no	Payload.
created_by_user_id	uuid	yes	Actor.
created_at	timestampz	no	Created.

Important indexes:

- index (clinic_id, created_at desc)

supported_languages

Platform-supported language registry.

Column	Type	Nullable	Purpose / notes
language_code	text	no	Primary key: ta_tanglish, english, hi, etc.
display_name	text	no	English display name.
native_name	text	yes	Native display.
enabled_platform_wide	boolean	no	Can clinics enable it?
default_tts_voice	text	yes	TTS voice code.
default_stt_language_hint	text	yes	STT hint.
fallback_language_code	text	yes	Fallback.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- primary key language_code

clinic_languages

Clinic-enabled languages and default.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
language_code	text	no	FK supported_languages.
is_default	boolean	no	Only one default per clinic.
enabled	boolean	no	Enabled for clinic.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique (clinic_id, language_code)
- partial unique one default per clinic where is_default=true

message_templates

Language-specific template registry.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
template_key	text	no	booking.ask_date etc.
language_code	text	no	FK supported_languages.
template_text	text	no	Template body with variables.
required_variables_json	jsonb	no	Required variables list.
active	boolean	no	Can be used.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique (template_key, language_code)

Implementation notes:

- Missing templates should fail loudly in QA and fallback safely in production.

language_pack_configs

Parser words and model hints per language.

Column	Type	Nullable	Purpose / notes
language_code	text	no	PK/FK supported_languages.
yes_words_json	jsonb	no	yes/seri/okay words.
no_words_json	jsonb	no	no/vendam/cancel words.
today_words_json	jsonb	no	today/inniku variants.
tomorrow_words_json	jsonb	no	tomorrow variants.
time_words_json	jsonb	no	morning/evening words.
classifier_examples_json	jsonb	no	Few-shot examples.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- primary key language_code

doctors

Clinical doctor profiles. May exist without login user.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
user_id	uuid	yes	Optional login user.
name	text	no	Doctor display name.
qualification	text	yes	Qualification.
registration_number	text	yes	Optional.
active	boolean	no	Can be routed/booked.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique (clinic_id, id)
- user_id optional

Important indexes:

- index (clinic_id, active)

Implementation notes:

- Doctor login link is through clinic_users.doctor_id; doctors.user_id is optional convenience.

clinic_services

Clinic-defined services Vaidya can route to.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
service_key	text	no	Unique within clinic.
service_name	text	no	Display name.
description	text	yes	Description.
handles_json	jsonb	no	Conditions/problems handled.
does_not_handle_json	jsonb	no	Conditions not handled.
red_flags_json	jsonb	no	Emergency/safety red flags.
routing_examples_json	jsonb	no	Examples for service router.
requires_staff_confirmation	boolean	no	If route should always be staff-confirmed.
active	boolean	no	Active service.

created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique (clinic_id, service_key)
- unique (clinic_id, id)

doctor_services

Many-to-many mapping of doctors to services, also stores doctor-service fees.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
doctor_id	uuid	no	FK doctors.
clinic_service_id	uuid	no	FK clinic_services.
consultation_fee_amount	numeric(12,2)	yes	Doctor-service consultation fee.
followup_fee_amount	numeric(12,2)	yes	Follow-up fee.
followup_valid_days	integer	yes	Follow-up validity.
active	boolean	no	Doctor handles service.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique (clinic_id, doctor_id, clinic_service_id)
- doctor and service must belong to same clinic

Important indexes:

- index (clinic_id, clinic_service_id, active)

doctor_service_booking_rules

Booking/slot-generation rule per doctor-service mapping.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
doctor_id	uuid	no	FK doctors.
clinic_service_id	uuid	no	FK clinic_services.
slot_duration_minutes	integer	no	Duration of generated slot window.
capacity_per_slot	integer	no	Number of patients per slot window.
booking_horizon_days	integer	no	Default 45.
min_booking_notice_minutes	integer	no	Minimum notice.
max_advance_booking_days	integer	yes	Optional cap.
manual_edit_cutoff_before_start_minutes	integer	no	Default 60.
manual_edit_max_shift_minutes	integer	no	Default 60.
effective_from	date	no	Rule start.
effective_to	date	yes	Rule end.
active	boolean	no	Active rule.
version	integer	no	Config version.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique active rule per clinic_id + doctor_id + clinic_service_id + effective range should be enforced in service logic or exclusion constraint

clinic_hours

Clinic regular working windows.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.

day_of_week	integer	no	0-6 or 1-7, choose one convention.
start_time	time	no	Opening window start.
end_time	time	no	Closing window end.
active	boolean	no	Active window.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Important indexes:

- index (clinic_id, day_of_week, active)

Implementation notes:

- PUT can replace complete weekly hours. Conflict check required.

clinic_holidays

Clinic holidays and partial closures.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
holiday_date	date	no	Holiday date.
is_full_day	boolean	no	Full-day or partial.
start_time	time	yes	Partial closure start.
end_time	time	yes	Partial closure end.
reason	text	yes	Reason.
active	boolean	no	Active holiday.
created_by_user_id	uuid	yes	Actor.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Important indexes:

- index (clinic_id, holiday_date, active)

Implementation notes:

- Adding holiday must not auto-cancel appointments. Conflict check first.

doctor_schedules

Doctor recurring availability windows.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
doctor_id	uuid	no	FK doctors.
day_of_week	integer	no	Weekly day.
start_time	time	no	Start.
end_time	time	no	End.
effective_from	date	yes	Optional start date.
effective_to	date	yes	Optional end date.
active	boolean	no	Active schedule.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Important indexes:

- index (clinic_id, doctor_id, day_of_week, active)

Implementation notes:

- Schedule changes require conflict check and slot regeneration.

doctor_blocked_slots

Doctor-specific blocks, leave, or unavailable windows.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.

clinic_id	uuid	no	FK clinics.
doctor_id	uuid	no	FK doctors.
start_time	timestampz	no	Block start.
end_time	timestampz	no	Block end.
reason	text	yes	Reason.
active	boolean	no	Active block.
created_by_user_id	uuid	yes	Actor.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- doctor belongs to same clinic

Important indexes:

- index (clinic_id, doctor_id, start_time, end_time)

Implementation notes:

- Adding block requires conflict check.

generated_slot_batches

Tracks slot generation/regeneration batches.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
trigger_source	text	no	daily_job, schedule_change, rule_change, holiday_change.
rule_id	uuid	yes	doctor_service_booking_rules id.
started_at	timestampz	no	Start.
completed_at	timestampz	yes	End.
status	text	no	running, completed, failed.
summary_json	jsonb	yes	Counts/conflicts.
created_at	timestampz	no	Created.

Important indexes:

- index (clinic_id, started_at desc)

appointment_slots

Generated doctor-service capacity time windows.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
doctor_id	uuid	no	FK doctors.
clinic_service_id	uuid	no	FK clinic_services.
start_time	timestampz	no	Slot window start.
end_time	timestampz	no	Slot window end.
capacity_total	integer	no	Total patients allowed in window.
status	text	no	open, blocked, cancelled, superseded.
generated_from_rule_id	uuid	yes	FK doctor_service_booking_rules.
generation_batch_id	uuid	yes	FK generated_slot_batches.
config_version	integer	yes	Rule version at generation.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- doctor and service must belong to clinic
- capacity_total > 0
- end_time > start_time

Important indexes:

- index (clinic_id, doctor_id, clinic_service_id, start_time, status)
- unique non-superseded slot identity by clinic/doctor/service/start/end

Implementation notes:

- Availability is derived by capacity_total minus appointments and holds.

slot_holds

Temporary reservations for selected slots during conversation/manual flow.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
slot_id	uuid	no	FK appointment_slots.
session_id	uuid	yes	FK conversation_sessions.
patient_phone	text	yes	Caller/patient phone.
status	text	no	active, expired, released, converted.
hold_expires_at	timestampz	no	Expiry.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- slot belongs to same clinic

Important indexes:

- index (clinic_id, slot_id, status, hold_expires_at)
- index (session_id, status)

Implementation notes:

- Active non-expired holds consume capacity.

patients

Clinic patient records.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
name	text	no	Patient name.
normalized_name	text	yes	Normalized for matching.
phone	text	yes	Phone.
normalized_phone	text	yes	Normalized phone.
date_of_birth	date	yes	Optional.
gender	text	yes	Optional.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique maybe (clinic_id, normalized_phone, normalized_name) when both present

Important indexes:

- index (clinic_id, normalized_phone)

Implementation notes:

- Phone alone is not unique because families share phone numbers.

appointment_requests

Actual appointment/request records.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
slot_id	uuid	yes	FK appointment_slots.
slot_hold_id	uuid	yes	FK slot_holds.
patient_id	uuid	yes	FK patients.

Vaidya Production LLD - Final Updated All Changes

patient_name	text	no	Captured name.
patient_phone	text	yes	Captured phone.
doctor_id	uuid	no	FK doctors.
clinic_service_id	uuid	no	FK clinic_services.
reason_for_visit	text	no	Problem/reason required.
normalized_reason	text	yes	Normalized for follow-up.
appointment_start	timestampz	no	Start.
appointment_end	timestampz	no	End.
status	text	no	pending_confirmation, confirmed, visited, cancelled, rescheduled, no_show, conflict_required.
is_followup	boolean	no	Follow-up flag.
followup_of_visit_id	uuid	yes	FK patient_visits.
routing_source	text	yes	service_router, returning_patient_followup, doctor_name, manual.
source_session_id	uuid	yes	conversation session.
replaces_appointment_id	uuid	yes	New appointment replaces old.
replaced_by_appointment_id	uuid	yes	Old appointment replaced by new.
created_by_user_id	uuid	yes	Admin/doctor if manual.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- doctor, service, slot must belong to clinic
- end > start
- reason_for_visit required

Important indexes:

- index (clinic_id, doctor_id, appointment_start)
- index (clinic_id, patient_phone, appointment_start)
- index (clinic_id, status, appointment_start)

Implementation notes:

- pending/confirmed consume slot capacity. cancelled/rescheduled do not.

appointment_action_requests

Patient-originated cancel/reschedule requests for admin action.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
appointment_id	uuid	no	FK appointment_requests.
request_type	text	no	cancel or reschedule.
requested_by	text	no	patient_call, admin, doctor.
status	text	no	pending, approved, rejected, completed.
requested_new_slot_id	uuid	yes	For reschedule.
requested_new_date	date	yes	If slot not selected.
requested_new_time_preference	text	yes	morning/evening etc.
reason	text	yes	Patient/admin reason.
source_call_id	uuid	yes	FK calls.
source_session_id	uuid	yes	FK conversation_sessions.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- appointment belongs to same clinic

Important indexes:

- index (clinic_id, status, created_at desc)

Implementation notes:

- Default patient cancel/reschedule by call creates this request; admin takes final action.

appointment_events

History of appointment lifecycle changes.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
appointment_request_id	uuid	no	FK appointment_requests.
event_type	text	no	created, confirmed, cancelled, rescheduled, marked_visited, conflict_required, notification_sent.
old_values_json	jsonb	yes	Before state.
new_values_json	jsonb	yes	After state.
actor_type	text	no	agent, clinic_admin, doctor, system, patient_call.
actor_user_id	uuid	yes	User actor.
source_session_id	uuid	yes	Conversation source.
created_at	timestampz	no	Created.

Important indexes:

- index (clinic_id, appointment_request_id, created_at)

patient_visits

Completed visit history used for follow-up routing.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
patient_id	uuid	no	FK patients.
appointment_request_id	uuid	yes	FK appointment_requests.
doctor_id	uuid	no	FK doctors.
clinic_service_id	uuid	no	FK clinic_services.
reason_for_visit	text	no	Visit reason required.
normalized_reason	text	yes	Normalized reason.
visited_at	timestampz	no	Visit timestamp.
followup_valid_until	timestampz	yes	Follow-up validity.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- doctor/service/patient belong to clinic

Important indexes:

- index (clinic_id, patient_id, visited_at desc)
- index (clinic_id, patient_id, clinic_service_id, normalized_reason)

conversation_sessions

Conversation state for text/voice/WhatsApp sessions.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
channel	text	no	web_demo, voice_call, whatsapp, admin_test.
patient_phone	text	yes	Caller/patient phone.
patient_id	uuid	yes	Matched patient.
language_code	text	no	Session language.
language_source	text	yes	clinic_default, patient_requested, detected.
current_flow	text	no	none, booking, cancel, reschedule, handoff, emergency, fee_clarification.
current_state	text	no	State machine state.
collected_json	jsonb	no	Slot filling and flow state.
status	text	no	active, completed, escalated, abandoned, expired.
expires_at	timestampz	yes	Session expiry.

created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Important indexes:

- index (clinic_id, patient_phone, created_at desc)

conversation_messages

Patient/assistant transcript at message-turn level.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
session_id	uuid	no	FK conversation_sessions.
sender	text	no	patient, assistant, system.
message_text	text	no	Message text.
intent	text	yes	Detected intent.
reply_template_key	text	yes	Template used for assistant.
flow_before	text	yes	Trace.
state_before	text	yes	Trace.
flow_after	text	yes	Trace.
state_after	text	yes	Trace.
debug_json	jsonb	yes	Dev only; do not expose in prod.
created_at	timestampz	no	Created.

Important indexes:

- index (clinic_id, session_id, created_at)

message_idempotency_keys

Prevents duplicate processing of message/API retries.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
session_id	uuid	yes	FK conversation_sessions.
idempotency_key	text	no	Client-provided key.
request_hash	text	yes	Hash to detect conflicts.
response_json	jsonb	yes	Cached response.
created_at	timestampz	no	Created.

Key constraints:

- unique (clinic_id, idempotency_key)

calls

Voice call metadata and outcome.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
conversation_session_id	uuid	yes	FK conversation_sessions.
provider	text	yes	exotel, twilio.
provider_call_id	text	yes	Provider id.
patient_phone	text	yes	Caller.
started_at	timestampz	yes	Start.
ended_at	timestampz	yes	End.
duration_seconds	integer	yes	Duration.
outcome	text	yes	appointment, callback, emergency, faq, no_answer, abandoned.
summary	text	yes	Call summary.
recording_object_key	text	yes	Object storage key.
recording_expires_at	timestampz	yes	10-day expiry.
recording_deleted_at	timestampz	yes	Deleted.
created_appointment_request_id	uuid	yes	Appointment link.
created_callback_request_id	uuid	yes	Callback link.

created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique provider_call_id per provider if present

Important indexes:

- index (clinic_id, started_at desc)
- index (clinic_id, outcome, started_at desc)

call_transcripts

Voice transcript turns retained for 30 days.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
call_id	uuid	no	FK calls.
speaker	text	no	patient, vaidya, system.
text	text	no	Transcript text.
started_at_ms	integer	yes	Audio offset.
ended_at_ms	integer	yes	Audio offset.
expires_at	timestampz	no	30-day expiry.
deleted_at	timestampz	yes	Deleted/anonymized.
created_at	timestampz	no	Created.

Important indexes:

- index (clinic_id, call_id, created_at)

recording_access_logs

Audit access to call recordings.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
call_id	uuid	no	FK calls.
accessed_by_user_id	uuid	no	User.
accessed_at	timestampz	no	Time.
ip_address	text	yes	IP.
user_agent	text	yes	User agent.

Important indexes:

- index (clinic_id, call_id, accessed_at desc)

callback_requests

Human callback requests.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
patient_name	text	yes	Name.
patient_phone	text	yes	Phone.
reason	text	yes	Reason.
status	text	no	pending, completed, cancelled.
source_session_id	uuid	yes	Conversation.
source_call_id	uuid	yes	Call.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Important indexes:

- index (clinic_id, status, created_at desc)

emergency_incidents

Emergency safety incidents detected by Vaidya.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
patient_phone	text	yes	Caller.
message_text	text	yes	Trigger text.
reason	text	yes	Reason/safety label.
source_session_id	uuid	yes	Conversation.
source_call_id	uuid	yes	Call.
status	text	no	open, reviewed, closed.
created_at	timestampz	no	Created.
reviewed_by_user_id	uuid	yes	Reviewer.
reviewed_at	timestampz	yes	Reviewed.

Important indexes:

- index (clinic_id, status, created_at desc)

knowledge_files

Uploaded knowledge/Q&A files.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
file_name	text	no	Original filename.
object_key	text	yes	Storage key.
file_type	text	yes	docx, pdf, xlsx.
status	text	no	uploaded, parsing, parsed, failed.
uploaded_by_user_id	uuid	yes	Uploader.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Important indexes:

- index (clinic_id, created_at desc)

clinic_knowledge_base

Canonical clinic-approved Q&A rows.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
knowledge_file_id	uuid	yes	Source file.
question	text	no	Question.
answer	text	yes	Canonical answer.
category	text	yes	parking, scan_prep, insurance, report, etc.
alternative_phrases_json	jsonb	no	Alternatives.
search_text	text	yes	Search text.
status	text	no	pending_review, approved, disabled, needs_update.
source_file	text	yes	Source display.
source_page	integer	yes	Page if PDF.
embedding	vector	yes	pgvector later.
approved_by_user_id	uuid	yes	Approver.
approved_at	timestampz	yes	Approval time.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Important indexes:

- index (clinic_id, status, category)
- full text / trigram indexes later
- vector index later when enabled

Implementation notes:

- Only status=approved can be used at runtime.

clinic_knowledge_translations

Approved translations of knowledge answers.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
knowledge_id	uuid	no	FK clinic_knowledge_base.
language_code	text	no	FK supported_languages.
question_text	text	yes	Translated question.
answer_text	text	no	Translated approved answer.
status	text	no	pending_review, approved, disabled.
approved_by_user_id	uuid	yes	Approver.
approved_at	timestampz	yes	Approved.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique (knowledge_id, language_code)

knowledge_answer_versions

Version history for approved Q&A edits.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
knowledge_id	uuid	no	FK clinic_knowledge_base.
version_number	integer	no	Version.
question	text	no	Question snapshot.
answer	text	yes	Answer snapshot.
status	text	no	Status at time.
changed_by_user_id	uuid	yes	Actor.
changed_at	timestampz	no	Time.

Key constraints:

- unique (knowledge_id, version_number)

notification_events

Async outbox for WhatsApp/SMS/email/internal notifications.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	no	FK clinics.
event_type	text	no	APPOINTMENT_CONFIRMED_PATIENT etc.
recipient_type	text	no	patient, admin, doctor, clinic_staff.
recipient_phone	text	yes	Phone.
recipient_user_id	uuid	yes	User.
channel	text	no	whatsapp, sms, email, internal.
template_key	text	yes	Provider/internal template.
payload_json	jsonb	no	Payload.
deduplication_key	text	yes	Prevents duplicates.
status	text	no	pending, processing, sent, failed, cancelled.
attempt_count	integer	no	Attempts.
next_attempt_at	timestampz	yes	Retry time.
provider_response_json	jsonb	yes	Response.
created_at	timestampz	no	Created.
updated_at	timestampz	no	Updated.

Key constraints:

- unique deduplication_key where not null

Important indexes:

- index (clinic_id, status, next_attempt_at)

audit_logs

Audit trail for critical actions.

Column	Type	Nullable	Purpose / notes
id	uuid	no	Primary key.
clinic_id	uuid	yes	Clinic if applicable.
actor_type	text	no	user, agent, system, provider.
actor_user_id	uuid	yes	User actor.
event_type	text	no	appointment_confirmed, agent_turned_off, etc.
resource_type	text	yes	appointment, slot, clinic_setting.
resource_id	uuid	yes	Resource id.
old_values_json	jsonb	yes	Before.
new_values_json	jsonb	yes	After.
metadata_json	jsonb	yes	Extra.
request_id	text	yes	Request id.
ip_address	text	yes	IP.
created_at	timestampz	no	Created.

Important indexes:

- index (clinic_id, event_type, created_at desc)
- index (resource_type, resource_id)

16.1 Critical database constraints and transaction rules

The exact migrations must include composite foreign keys and indexes that enforce clinic scoping. The following SQL-style snippets describe the required approach.

```
-- Composite foreign keys pattern
-- doctors and clinic_services should expose unique (clinic_id, id).
-- appointment_slots should reference (clinic_id, doctor_id) and (clinic_id, clinic_service_id).

-- Slot hold transaction must SELECT appointment_slots FOR UPDATE before counting active
appointments and holds.

-- Avoid hard delete. Use statuses: active=false, cancelled, disabled, expired, superseded.
```

17. API Contract Summary

Detailed OpenAPI should be generated separately, but this section defines implementation direction. Every write API must validate RBAC, clinic_id, idempotency, and business rules. Production must not expose debug fields.

API area	Endpoints
Auth	POST /v1/auth/request-otp; POST /v1/auth/verify-otp; POST /v1/auth/logout; GET /v1/me
Conversation	POST /v1/conversations; POST /v1/conversations/{sessionId}/messages; GET /v1/conversations/{sessionId}; PATCH /v1/conversations/{sessionId}/language
Clinic settings	GET /v1/clinic/settings; PATCH /v1/clinic/settings; optional PUT /v1/clinic/settings
Doctors	GET/POST /v1/doctors; GET/PATCH /v1/doctors/{doctorId}; POST /v1/doctors/{doctorId}/disable
Doctor schedules	GET /v1/doctors/{doctorId}/schedules; PUT full; POST add; PATCH /v1/doctors/{doctorId}/schedules/{scheduleId}; POST disable
Clinic hours	GET /v1/clinic-hours; PUT full; POST add; PATCH /v1/clinic-hours/{id}; POST disable
Holidays	GET/POST /v1/holidays; PATCH /v1/holidays/{id}; POST /v1/holidays/{id}/disable
Services	GET/POST /v1/clinic-services; PATCH/disable service;

Booking rules	GET/POST/PATCH/disable doctor-services GET/PUT/PATCH /v1/doctors/{doctorId}/services/{serviceId}/booking-rule
Appointments	GET /v1/appointments; POST /v1/appointments/manual; POST confirm/cancel/reschedule/mark-visited; POST quick-edit
Action requests	GET /v1/appointment-action-requests; POST approve/reject
Knowledge	GET /v1/knowledge; POST manual; POST upload-docx; POST approve/bulk-approve; PATCH edit; POST disable
Calls	GET /v1/calls; GET /v1/calls/{id}; GET recording-url; GET transcript
Notifications	GET /v1/notification-events; POST test later
Languages	GET /v1/languages; GET/PUT /v1/clinic/languages
Subscriptions	GET /v1/subscription/plans; GET/PATCH /v1/clinic/subscription; GET /v1/clinic/usage

17.1 Standard error response

```
{
  "error": {
    "code": "SLOT_NOT_AVAILABLE",
    "message": "Selected slot is no longer available.",
    "details": { "slot_id": "...",
    "request_id": "req_..."
  }
}
```

- Common error codes: UNAUTHORIZED, FORBIDDEN, CLINIC_NOT_FOUND, DOCTOR_NOT_OWNER, SLOT_NOT_AVAILABLE, SLOT_HOLD_EXPIRED, APPOINTMENT_NOT_CONFIRMABLE, APPOINTMENT_NOT_CANCELLABLE, CONFLICTING_APPOINTMENTS, KNOWLEDGE_NOT_APPROVED, VALIDATION_ERROR, IDEMPOTENCY_CONFLICT, PROVIDER_FAILURE.

18. State Transitions, Test Matrix, and Readiness Checklist

18.1 Appointment status transitions

From	Allowed to
pending_confirmation	confirmed, cancelled, rescheduled, conflict_required
confirmed	visited, cancelled, rescheduled, no_show, conflict_required
conflict_required	confirmed, cancelled, rescheduled
cancelled	terminal
visited	terminal
no_show	terminal
rescheduled	terminal or replaced record, based on cancel/rebook model

18.2 Slot/hold status transitions

Entity	Transitions
appointment_slots	open -> blocked/cancelled/superseded; blocked -> open; superseded terminal for runtime booking.
slot_holds	active -> converted; active -> expired; active -> released; converted/expired/released terminal.

18.3 Golden tests to add

- Capacity > 1 slot allows N appointments/holds and rejects N+1.
- Two patients choose same capacity=1 slot concurrently: only one hold succeeds.
- Hold expires and capacity becomes available.
- Patient confirms after hold expiry: backend re-checks capacity.
- Cancelled appointment releases capacity.
- Capacity decrease with no conflict updates future open slots.
- Capacity decrease with conflict blocks change and returns conflict list.

- Slot duration increase with no conflict supersedes old slots and creates new slots.
- Slot duration change with conflict is blocked.
- Daily SlotGenerationJob maintains 45-day horizon without duplicates.
- Doctor quick edit blocked within cutoff.
- More-than-1-hour time change creates formal cancel/rebook/reschedule flow.
- Admin outside-hours manual appointment requires override_reason.
- Clinic admin disables doctor login without disabling doctor profile.
- Agent off forwards to fallback_phone.
- Knowledge structured-vs-KB precedence works.
- New language can be added by templates/parser config without state-machine changes.

18.4 Production readiness checklist

- All APIs require clinic_id scoping and RBAC checks.
- LLM cannot access repositories or DB clients.
- All writes have ActionValidator and audit log.
- Slot holds and appointment creation use DB transactions.
- notification_events outbox is used for WhatsApp/SMS/emergency/callback alerts.
- Recording signed URLs expire and access is audited.
- Audio cleanup and transcript cleanup jobs are running.
- Dev auth and debug fields disabled in production.
- QA and production databases separated.
- Migrations reviewed and rollback plan exists.
- Provider credentials stored securely.
- Plan limits enforced for concurrent calls and retention.

19. Implementation Priority

Priority	Build items
P0	DB schema/migrations, Auth/RBAC foundation, clinic/settings/users/doctors/services/schedules/booking rules, appointment slot generation, manual appointment, appointment capacity engine.
P1	Text agent, booking flow, side questions, knowledge Q&A, pending confirmation workflow, doctor/admin appointment screens.
P2	Cancel/reschedule action requests, notifications outbox, call inbox placeholders, DOCX knowledge parsing.
P3	Voice adapter, recordings, transcripts, emergency alert delivery, provider integration.
P4	Subscriptions/billing UI, advanced language expansion, embeddings/semantic search, reports, analytics.

This document should be paired with actual migration files, OpenAPI/DTO files, template registry JSON/TS, and frontend screen tickets before full coding starts.